4.3 Class Design

The class design of Orbit organises browser control, prompt-injection detection and AI assistance around TypeScript service classes. The React interface and several supporting operations use functions, while TypeScript interfaces define exchanged data. The accompanying diagram shows the principal classes and their relationships, identifying the security route module separately.

BrowserRuntime maintains registered browsing targets and temporary element handles. Its registerTarget(), unregisterTarget() and invoke() operations manage guest targets and dispatch commands. CdpSession represents a guest-page connection through the Chrome DevTools Protocol. CdpSessionRegistry maintains one shared session per guest for BrowserRuntime and CdpInspectionService. The inspection service maintains capture state and provides watch(), forget() and capture() operations to collect webpage content for user-initiated scans.

BrowserUIController manages menu surfaces, active downloads and zoom information. It owns a BrowserProfileStore containing history, bookmarks, download records and settings. The store provides recordVisit(), saveBookmark(), updateSettings() and snapshot(), while flush() saves the profile in a local JSON file. The React browser shell accesses these classes through the preload bridge.

PromptPreprocessingService removes HTML where applicable and normalises case and whitespace. TextChunkingService divides text into identified, overlapping sections. The security route module coordinates these services and calls PromptClassifierService, which maintains model readiness and combines RuleBasedDetectorService with local Prompt Guard 2 inference. The rule detector returns suspicious patterns and matched evidence. Either detector can flag a chunk as malicious; if the model cannot load, classification continues with rules.

FeatureExplanationService derives supporting evidence from text and matched phrases. The security route module combines this evidence with chunk results and the final decision for the analysis panels. These results are structured data contracts. The classifier also checks the chat prompt before LlmProviderManager forwards the request. This check covers the prompt field; optional page context, history and attachments are not automatically classified on that path.

AgentTask coordinates Agent Mode and holds the task identifier, goal, target identifier, execution limit and progress. Each task creates one AgentWorkingMemory object to record actions, failures, retries, findings and milestones. Its snapshot() supplies bounded context to the planner. AgentTask also uses the shared AgentSessionMemory instance for recent terminated-task summaries during the renderer's lifetime.

AgentPlannerService receives the goal, memory snapshot and page state through the local planning endpoint. Its buildMessages(), requestPlan() and parsePlan() operations prepare model input and validate the action queue through the tool registry. AgentTask applies approval checks and invokes BrowserRuntime through interprocess communication. The runtime executes commands and returns observations or verification results. This planning path does not automatically classify the goal or page observations with PromptClassifierService.

ProviderGateway is the abstract class for AI-provider access. OpenAICompatibleGateway, AnthropicGateway and GeminiGateway implement listModels(), chatCompletion() and validateKey(). LlmProviderManager holds the active configuration and gateway, allowing chat and planning to share the provider contract. A factory selects the concrete gateway from the configuration. This is the main inheritance relationship; browser, detection and agent classes cooperate through ownership, shared references and service dependencies.